

1. RSA-Related Tools

rsatool

Repository: <https://github.com/ius/rsatool.git>

Purpose: - Used for working with RSA keys and parameters. - Can generate RSA keys, calculate missing RSA values, and assist in RSA-related CTF challenges. - Helpful when partial RSA parameters (p, q, n, e, d) are known.

RsaCtfTool

Repository: <https://github.com/RsaCtfTool/RsaCtfTool.git>

Purpose: - Advanced tool for solving RSA-based CTF challenges. - Automates multiple RSA attacks such as weak keys, common modulus, Wiener's attack, and factorization-based attacks. - Widely used in cryptography and CTF competitions.

2. Hash Identification Tools

hashID

Purpose: - Identifies the possible hash algorithm based on a given hash string. - Provides multiple possible hash types with confidence levels. - Useful as a first step before cracking a hash.

hashes.com

Website: <https://hashes.com>

Purpose: - Online hash lookup and cracking service. - Uses large databases of previously cracked hashes. - Helpful for quick verification or identification of common hashes.

3. Hashcat (Password Cracking Tool)

What is Hashcat?

- Hashcat is a powerful and fast password recovery tool.
- Supports multiple hash algorithms and attack modes.
- Uses CPU and GPU acceleration for high performance.

Basic Hashcat Syntax

```
hashcat -m <hash_type> -a <attack_mode> hashfile wordlist
```

Explanation of Parameters

- `-m <hash_type>` : Specifies the hash type in numeric format.
Example: `-m 1000` → NTLM hash
Use `man hashcat` or the official documentation to find hash-type codes.
- `-a <attack_mode>` : Specifies the attack mode.
Example: `-a 0` → Straight attack (tries passwords one by one from a wordlist).
- `hashfile` : File containing the hash(es) to be cracked.
- `wordlist` : Password list used during the attack.

Example Command

```
hashcat -m 3200 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
```

Explanation: - Treats the hash as a Bcrypt hash (`-m 3200`). - Uses a straight attack (`-a 0`). - Attempts passwords from the `rockyou.txt` wordlist.

Reference

- Example hashes and hash-type IDs:
https://hashcat.net/wiki/doku.php?id=example_hashes

4. Hashing Concepts

What is Hashing?

- Hashing is a one-way process that converts input data into a fixed-size output called a **hash** or **digest**.
- Even a small change in input data results in a completely different hash.
- Hashing cannot be reversed to obtain the original data.

Common Uses: - Password storage - Data integrity verification - Digital signatures

5. Encoding vs Hashing vs Encryption

Encoding

- Converts data from one format to another for compatibility.
- Reversible process.
- Common encoding formats:

- ASCII
- UTF-8
- UTF-16
- UTF-32
- ISO-8859-1
- Windows-1252

UTF-8, UTF-16, and UTF-32 are Unicode encodings that support multiple languages such as Arabic and Japanese.

Hashing

- One-way process.
- Cannot retrieve original data.
- Used for verification and integrity.

Symmetric Encryption

- Uses the **same secret key** for both encryption and decryption.
- Also called **private-key cryptography**.

How it works: 1. Plaintext is encrypted using a secret key. 2. The same key is used to decrypt the ciphertext back into plaintext.

Common Algorithms: - AES (Advanced Encryption Standard) - DES (Data Encryption Standard – deprecated) - 3DES - Blowfish - ChaCha20

Use Cases: - Disk encryption (BitLocker, FileVault) - Secure file storage - VPNs and secure communications

Challenges: - Secure key distribution is difficult. - Key management becomes complex with many users. - If the key is compromised, all encrypted data is exposed. - Vulnerable in high-risk environments (e.g., industrial espionage).

Asymmetric Encryption

- Uses **two different keys**:
- **Public key** (shared openly)
- **Private key** (kept secret)
- Also called **public-key cryptography**.

How it works: 1. Data encrypted with a public key can only be decrypted using the corresponding private key. 2. Data encrypted with a private key can be verified using the public key (digital signatures).

Common Algorithms: - RSA - ECC (Elliptic Curve Cryptography) - DSA - ElGamal

Use Cases: - Secure key exchange - Digital signatures - SSL/TLS (HTTPS) - Email encryption (PGP, S/MIME)

Advantages: - No need to share private keys. - More secure key distribution.

Disadvantages: - Slower than symmetric encryption. - Not suitable for encrypting large amounts of data.

Hybrid Encryption

- Combines **symmetric** and **asymmetric** encryption.

How it works: 1. Asymmetric encryption is used to securely exchange a symmetric session key. 2. Symmetric encryption is then used for fast data transfer.

Examples: - HTTPS (TLS handshake) - Secure messaging apps

Encryption vs Hashing vs Encoding (Quick Comparison)

Feature	Encryption	Hashing	Encoding
Reversible	Yes	No	Yes
Uses Key	Yes	No	No
Purpose	Confidentiality	Integrity	Compatibility
Example	AES, RSA	SHA-256	Base64, UTF-8

6. Summary

- **rsatool & RsaCtfTool:** Used for RSA cryptography analysis and CTF challenges.
- **hashID & hashes.com:** Used for identifying hash types.
- **Hashcat:** Used for cracking hashes using wordlists and attack modes.
- **Hashing $\neq$ Encoding $\neq$ Encryption:** Each serves a different security purpose and should not be confused.